

Introduction to NLP

Jacques Bahi

- 1 Definition and Key Concepts
- 2 History and Evolution of NLP
- 3 Applications of NLP in Various Domains
- 4 Foundations of NLP
 - Computational Linguistics
 - Statistics Applied to NLP
- 5 Computer Science and NLP
- 6 Text Normalization
- 7 Text Vectorization
- 8 Review of some useful operations on strings

- 9 High-level overview of the spaCy library
- 10 Installation of spaCy
- 11 Basic Operations with spaCy
- 12 Overview of spaCy conventions
- 13 Introduction to Tokenization
- 14 Customizing the tokenizer
- 15 Debugging the tokenizer
- 16 Sentence segmentation
- 17 Understanding lemmatization
- 18 Lemmatization in Natural Language Understanding (NLU)

19 spaCy Container Objects

- Token
- Span

20 spaCy Linguistic Features

21 POS

22 NER

23 Rule-based matching using spaCy

24 Dependency Analysis

25 Tree and Subtree Navigation

26 Text Data Cleaning

- Removing Stop Words
- Word Frequency
- Preprocessing Functions

Definition and Key Concepts I

- ▶ Natural Language Processing (NLP) is a branch of artificial intelligence (AI) that focuses on understanding, interpreting, and manipulating human language by machines. NLP encompasses domains such as linguistics, computer science, and AI to create tools capable of interpreting and synthesizing text for various applications.
- ▶ The goal of NLP is to enable computers to understand text and spoken words in the same way as humans do, by analyzing the grammatical structure of sentences and the individual meaning of words, and then using algorithms to extract the overall meaning of the text and provide results to user queries.

Definition and Key Concepts II

- ▶ Applications of NLP include automatic translation, voice command response, large-scale text synthesis, and natural language response generation.
- ▶ Introduction to NLP requires an understanding of key concepts underlying this field. One of the essential tasks of NLP is Automatic Speech Recognition (ASR), which involves converting speech into text. This technology is used in transcription systems, voice assistants, and many other applications like those developed by Google, Apple, or Amazon.

Definition and Key Concepts III

- ▶ Another essential task is Natural Language Understanding (NLU), which is crucial to enable machines to understand the meaning and intention behind human language. It involves syntactic, semantic, and pragmatic analysis of text to extract useful information and make appropriate decisions.
- ▶ Another key component of NLP is Natural Language Generation (NLG), which involves producing text or human language from structured data or previously acquired information. This can be used in applications such as automated report writing, summary generation, and interactive dialogues.

Definition and Key Concepts IV

- ▶ NLP also relies on machine learning techniques such as supervised and unsupervised learning to train language models capable of understanding and generating text. These models often rely on deep neural networks, such as recurrent neural networks (RNNs) and transformers, which can capture the structure and relationships of language.
- ▶ Sentiment analysis is another important application of NLP, which determines the opinion, emotion, or attitude expressed in text. This is used in the marketing field to assess customer perception of a product or brand, as well as in strategic intelligence to track public opinion on specific topics.

Definition and Key Concepts V

- ▶ Finally, NLP encompasses other areas such as automatic translation, context-based natural language understanding, Named Entity Recognition (NER) to extract specific information, coreference resolution to identify relationships between different parts of the text, and many others.
- ▶ NLP is a multidisciplinary field that combines techniques from AI, linguistics, and machine learning to enable machines to understand and generate human language accurately and efficiently. It plays a crucial role in various applications, from text data analysis

Definition and Key Concepts VI

to human-machine interaction, and continues to evolve rapidly with new technological advancements.

In summary, NLP is important as it facilitates communication between humans and machines, allowing computers to process and analyze textual and vocal data effectively. This can help businesses automate tasks, analyze data for valuable insights, and improve the quality of analyses. Applications of NLP are numerous and include chatbots, voice assistants, search engines, automatic translation, sentiment analysis, and many more.

History and Evolution of NLP I

- ▶ The development of Natural Language Processing (NLP) dates back to the 1950s when computer scientists started exploring the possibility of interacting machines with human language. At that time, early work focused on tasks such as automatic translation and automatic speech recognition
- ▶ In the 1980s and 1990s, the advent of statistical approaches provided a significant boost to NLP. Researchers began using probabilistic models to enhance language understanding, including techniques such as Hidden Markov Models (HMMs) and n-gram-based language

History and Evolution of NLP II

models. This led to improved results in tasks such as automatic speech recognition and automatic translation.

► In the early 21st century, the emergence of machine learning and neural networks revolutionized the field of NLP. The use of deep neural networks, such as recurrent neural networks (RNNs) and transformers, enabled more efficient capturing of language structure and relationships. Neural network-based language models achieved remarkable results in complex tasks such as automatic translation, text generation, and natural language understanding.

History and Evolution of NLP III

- ▶ Another key milestone in the evolution of NLP was the explosion of data and the rise of Big Data. Modern NLP models have benefited from access to vast amounts of textual data, allowing them to learn from a considerable volume of examples and improve their accuracy and generalization. Advances in parallel processing and computing power have also contributed to speeding up NLP tasks, enabling real-time and large-scale processing.
- ▶ In recent years, NLP has experienced significant advancements with the emergence of pretrained language models such as BERT (Bidirectional Encoder Representations from Transformers) and GPT (Generative Pretrained Transformer). These models are trained

History and Evolution of NLP IV

on large-scale datasets and are capable of capturing high-level semantic and syntactic information. They have achieved outstanding performance in various NLP tasks, while reducing the need for task-specific training.

► Practical applications of NLP have multiplied in various domains such as virtual assistants, information retrieval, sentiment analysis, fake news detection, context-based natural language understanding, and more. NLP continues to evolve rapidly, with constant

History and Evolution of NLP V

advances in the areas of human language understanding, text generation, and human-machine interaction, opening new possibilities for artificial intelligence and language-based systems.

Applications of NLP in Various Domains I

- ▶ NLP offers numerous applications in various domains. Firstly, in the field of virtual assistance, NLP is used to develop intelligent personal assistants such as Siri, Alexa, and Google Assistant. These systems utilize automatic **speech recognition and language understanding** to respond to user queries, schedule events, provide information, and execute voice commands.
- ▶ In the field of information retrieval, NLP is used to extract relevant information from vast volumes of textual data. This facilitates efficient search and **information retrieval** from documents, news articles, databases, social media, and more. Engineers can leverage

Applications of NLP in Various Domains II

these techniques to develop effective information retrieval systems and enhance access to knowledge.

- ▶ NLP also finds applications in **sentiment analysis** and **opinion mining**. It enables the analysis of sentiments and opinions expressed in social media, product reviews, user comments, etc. This analysis is used in the marketing field to assess customer perception of a product or brand, adapt communication strategies, and make informed decisions.
- ▶ In the healthcare domain, NLP is used to extract information from electronic medical records, laboratory reports, and scientific publications. It facilitates the retrieval of

Applications of NLP in Various Domains III

relevant medical information, symptom analysis, disease detection, and clinical decision support. Researchers and engineers interested in the healthcare field can harness NLP to develop medical assistance systems and clinical data analysis tools.

► NLP is also used in the finance domain, where it enables the analysis of financial reports, market trends tracking, fraud detection, and automation of financial document processing. Engineers can apply NLP to develop financial analysis and language-based forecasting systems.

Applications of NLP in Various Domains IV

► In the education field, NLP can be used to develop intelligent tutoring systems, adaptive learning platforms, and automated assessment tools for written responses. NLP allows the analysis of students' responses, provides personalized feedback, and adapts learning based on individual needs.

Applications of NLP in Various Domains V

In summary, NLP offers exciting applications in domains such as virtual assistance, information retrieval, sentiment analysis, healthcare, finance, and education.

Researchers and engineers can leverage NLP techniques to develop innovative systems and contribute to advancements in various sectors through artificial intelligence and human language analysis.

Foundations of NLP I

NLP is a multidisciplinary field where several specialized domains interact to build an operational and efficient NLP model.

Computational Linguistics I

- ▶ Computational linguistics is one of the foundations of Natural Language Processing (NLP). It studies the structures and rules of human language with the aim of formalizing and applying them to computer systems. Knowledge of linguistic principles is essential to understand and analyze language accurately.
- ▶ Within the scope of computational linguistics, several key concepts are addressed. First, **morphology** examines the structure of words and the rules of formation, such as prefixes, suffixes, and roots. This allows understanding how words are formed and applying morphological analysis techniques in NLP.

Computational Linguistics II

- ▶ Next, **syntax** focuses on the grammatical structure of sentences and the rules governing the relationships between words. It studies concepts such as grammatical categories, syntactic constituents, and dependency trees. Knowledge of syntax is essential for syntactic analysis and understanding the grammatical relationships in NLP.
- ▶ **Semantics**, on the other hand, examines the meaning of words and phrases. It investigates semantic relations such as synonymy, antonymy, hyponymy, hypernymy, and predications. Understanding semantics is crucial for text interpretation and generation in NLP.

Computational Linguistics III

► Lastly, **pragmatics** studies the use of language in a given context, taking into account intentions, inferences, and implicatures. It deals with aspects such as reference, coherence, presupposition, and speech act theory. Pragmatics is essential for understanding the meaning of language in real situations and for interacting appropriately with users in NLP applications.

Statistics Applied to NLP I

- ▶ Statistics play a crucial role in NLP by providing methods and models for processing and analyzing textual data. The application of statistics to NLP enables information extraction, learning from large datasets, and making decisions based on probabilities.
- ▶ In the field of NLP, statistics are used for tasks such as automatic speech recognition, machine translation, sentiment analysis, and information extraction. Statistical models such as Hidden Markov Models (HMMs), n-gram language models, and probabilistic models are used to capture language structures and trends.

Statistics Applied to NLP II

- ▶ Machine learning is also an important component of statistics applied to NLP. Supervised and unsupervised learning techniques are used to train NLP models capable of understanding and generating text. Methods such as classification, regression, clustering, and neural networks enable the development of high-performance NLP models for different tasks.
- ▶ Furthermore, statistics provide tools for evaluating and measuring the performance of NLP systems. Measures such as precision, recall, F-measure, and perplexity are used to assess the quality of results obtained by NLP models.

Computer Science and NLP I

- ▶ Computer science plays a central role in the field of Natural Language Processing. It provides the tools and techniques necessary for processing, storing, representing, and manipulating textual data efficiently.
- ▶ In NLP, data preprocessing techniques are essential for **cleaning** and **normalizing** texts by removing unwanted characters, converting to lowercase, lemmatizing, and eliminating stop words. This prepares textual data for subsequent analysis and processing.
- ▶ Data structures such as lists, arrays, trees, and graphs are used to represent textual information in an organized manner.

Computer Science and NLP II

Search algorithms and graph traversal algorithms are used to extract and manipulate information in textual data.

- ▶ Computer science also offers machine learning techniques for developing high-performance NLP models. Libraries and frameworks such as **TensorFlow**, **PyTorch**, **Scikit-learn** provide tools for training and deploying large-scale NLP models.
- ▶ Finally, software engineering is crucial in the development of NLP systems. Concepts such as modularity, reusability, error handling, and performance optimization are important for building robust and scalable NLP systems.

Computer Science and NLP III

In conclusion, the foundations of NLP are based on computational linguistics, statistics applied to NLP, and computer science. A deep understanding of these concepts is essential for developing effective and innovative NLP systems.

Text Normalization I

Text normalization in NLP is a crucial step in preparing textual data for analysis and modeling by transforming the text into a unique canonical form to reduce variability and facilitate subsequent analyses and modeling. Here are some commonly used text normalization techniques in NLP :

- Converting to Lowercase :

Converting all uppercase letters to lowercase to standardize the text and avoid treating words as different due to case sensitivity.

Text Normalization II

- Removing Accents

Removing accents from characters to reduce word variability and facilitate processing.

- Expanding Contractions :

Expanding contractions (e.g., "I've" to "I have") so that words are represented in their full form.

Text Normalization III

- Spelling Correction

Correcting spelling mistakes to reduce word variability and improve the quality of analyses.

- Removing Special Characters

Removing special characters such as emoticons and symbols to reduce the size of the vocabulary.

Text Normalization IV

Text normalization is an important step in NLP data preprocessing as it reduces the variability of words and facilitates subsequent analyses and modeling. Once the text is normalized, it can undergo further preprocessing steps such as tokenization, stop word removal, stemming, and lemmatization.

Text Vectorization I

Text vectorization is a fundamental step in Natural Language Processing (NLP) that allows representing textual data in the form of **numerical vectors**, understandable by machine learning algorithms. This transformation is necessary because machine learning models cannot directly process textual data.

There are several approaches to text vectorization in NLP. One of the most commonly used methods is the **bag-of-words** representation.

In this approach, each document is represented as a vector whose dimensions correspond to the words present in the corpus.

Text Vectorization II

Each element of the vector represents the frequency of occurrence of the corresponding word in the document. However, this method ignores the order of words and does not capture the relationships between them.

A more advanced approach is the **weighted bag-of-words representation (TF-IDF)**. This method takes into account the frequency of occurrence of words in a specific document, as well as their inverse frequency in the entire corpus.

This allows assigning greater weights to words specific to a document and reducing the importance of frequent and non-discriminating words.

Text Vectorization III

Another popular method is the word embeddings representation, such as Word2Vec and GloVe. These techniques use machine learning models to create dense vector representations of words.

Each word is represented by a vector of real numbers, where similar words are close in the vector space.

This approach captures the semantic and syntactic relationships between words and allows capturing richer information about the text.

Text Vectorization IV

More recently, pre-trained language models, such as BERT (Bidirectional Encoder Representations from Transformers), have revolutionized text vectorization. These models are trained on vast textual corpora and are capable of generating vector representations of words, sentences, and even entire documents. The vector representations obtained from these models contain high-level semantic and syntactic information, which allows achieving remarkable performance in many NLP tasks.

Text Vectorization V

Text vectorization in NLP plays an essential role in various tasks, such as text classification, sentiment analysis, information retrieval, and text generation. It allows transforming text into a numerical format suitable for machine learning, thus facilitating the analysis and extraction of knowledge from textual data. By choosing the appropriate vectorization method, taking into account the specifics of the task and the text corpus, it is possible to obtain effective vector representations that capture the relevant information for text analysis.

Review of some useful operations on strings I

In Python, text is represented by strings, objects of the `str` class. Strings are immutable sequences of characters.

► Creating a string object is easy - we surround the text with quotes :

```
word = 'Hello World'
```

Now, the `word` variable contains the string `Hello World`. As we mentioned, strings are sequences of characters, so we can ask for the first element of the sequence :

```
1 print (word [0])
2 H
3 # We can in the same way access other indices, as long as the index
  does not exceed the bounds:
4 word [4]
5 'o'
6 # We can use the len method, for length:
7 len(word)
8 11
9 # We can also iterate over the characters of a string with sequence
  methods:
10 for ch in word:
11     print(ch)
```

```
12 H
```

```
13 e
```

```
14 l
```

```
15 l
```

```
16 o
```

```
17 W
```

```
18 o
```

```
19 r
```

```
20 l
```

```
21 d
```

```
22
```

```
23 # Let's now review other string methods like counting characters,  
    searching for a substring, and changing case.
```

```
24 count counts the number of occurrences of a character in the string,  
    so the output is 3 here:  
25 word.count('l')  
26 3  
27 # Often, you need to find the index of a character for a certain  
    number of substring operations like slicing and dicing the string:  
28 word.index(e)  
29 1  
30 # Similarly, we can search for substrings in a string with the find  
    method:  
31 word.find('World')  
32 6  
33 # find returns -1 if the substring is not in the string:
```

```
34 word.find('Bonjour')
35 -1
36 # Searching for the last occurrence of a substring is also easy:
37 word.rfind('l')
38 9
39 # We can change the case of letters by the upper and lower methods:
40 word.upper()
41 'HELLO WORLD'
42 # The upper method changes all characters to uppercase. Similarly, the
    lower method changes all characters to lowercase:
43 word.lower()
44 'hello world'
45 # The capitalize method capitalizes the first character of the string:
```

```
46 'hello madam'.capitalize()
47 'Hello madam'
48 # The title method puts the string in title case. Title literally
    means making a title, so every word of the string is capitalized:
49 'hello madam'.title()
50 'Hello Madam'
51 Forming new strings from other strings can be done in several ways. We
    can concatenate two strings by adding them:
52 'Hello Madam!' + 'Have a nice day.'
53 'Hello Madam!Have a nice day.'
54 # We can also multiply a string by an integer. The output will be the
    string concatenated to itself by the number of times specified by
    the integer:
```

```
55 'sweet ' * 5
56 'sweet sweet sweet sweet sweet '
57
58 # join is a frequently used method; it takes a list of strings and
   joins them into one string
59
60 ' '.join(['hello', 'madam'])
61 'hello madam'
```

There exists a variety of substring methods. Replacing a substring means changing all its occurrences with another string :

```
1 'hello madam'.replace('hello', 'good morning')
2 'good morning madam'
```

Obtaining a substring by index is called slicing. You can slice a string by specifying the start and the end index.

```
1 word = 'Hello Madam Flower'
2 word [6:11]
3 'Madam'
```

Getting the first. Leaving the first index empty *rightarrow* the index starts at zero :

```
1 word [:5]
2 'Hello'
3 # Leaving the second index empty also has a special meaning - it means
   the rest of the string:
4 word [12:]
5 'Flower'
```

High-level overview of the spaCy library I

- ▶ spaCy is an open-source Python library for modern NLP.
- ▶ spaCy comes with pre-trained language models and word vectors for over 60 languages.
- ▶ spaCy is focused on production and delivering code, unlike its more academic predecessors.
- ▶ The most famous and frequently used Python predecessor is NLTK. The main goal of NLTK was to give students and researchers an idea of language processing. It never claimed to be efficient.

High-level overview of the spaCy library II

► spaCy focused on providing production-ready code from day one. You can expect the models to work on real-world data, the code to be efficient and capable of processing a huge amount of text data in a reasonable time.

You can find spaCy's features and performances on its website. Here is a simple benchmark

High-level overview of the spaCy library III

Name	Spark NLP	spaCy	NLTK	CoreNLP
Sentence detection	Yes	Yes	Yes	Yes
Tokenization	Yes	Yes	Yes	Yes
Stemming	Yes	Yes	Yes	Yes
Lemmatization	Yes	Yes	Yes	Yes
POS tagger	Yes	Yes	Yes	Yes
NER	Yes	Yes	Yes	Yes
Dependency parse	Yes	Yes	Yes	Yes
Text matcher	Yes	Yes	No	Yes
Date matcher	Yes	No	No	Yes
Chunking	Yes	Yes	Yes	Yes
Spell checker	Yes	No	No	No
Sentiment detector	Yes	No	No	Yes
Pretrained models	Yes	Yes	Yes	Yes

- ▶ spaCy's code is also professionally maintained.
- ▶ You can also check out spaCy's GitHub repository at <https://github.com/explosion/spaCy>, report a bug, or ask for help from the community.
- ▶ Another predecessor is CoreNLP (also known as StanfordNLP). CoreNLP is implemented in Java. Although CoreNLP is competitive in terms of efficiency, Python won thanks to the ease of prototyping and spaCy is much more professional as a software package.
- ▶ We will use the latest versions of spaCy for all our computational linguistics and machine learning needs.

Here are the features of the latest version :

- Tokenization preserving original data.
- Statistical sentence segmentation.

- Named Entity Recognition (NER)
- Part-of-Speech (POS) tagging.
- Dependency parsing.
- Pre-trained word vectors.
- Easy integration with popular deep learning libraries. PyTorch, TensorFlow, and MXNet.

Installation of spaCy I

■ Installation of spaCy with pip

You can install spaCy with the following command :

```
1 pip install spacy
```

```
2
```

■ Updating spaCy version to the latest

```
1 pip install -U spacy
```

```
2
```

■ Installation of spaCy with conda

The conda support is provided by the conda community. The command to install spaCy with conda is the following :

```
1 conda install -c conda-forge spacy
```

```
2
```

■ Checking the version you have :

```
1 python -m spacy info
2
```

Installation of spaCy on Windows

► If you have a Windows system, you need to install a version of Visual C++ Build Tools or Visual Studio Express that matches your Python distribution.

You should be able to import spaCy into your Python shell with the command : `import spacy`

Basic Operations with spaCy I

► In this chapter, you will learn the basic operations with spaCy, such as creating a language pipeline, tokenizing the text, and splitting the text into sentences.

First, you will learn what a language processing pipeline is and the components of the pipeline. We will continue with the general conventions of spaCy - important classes and organization of classes - to help you better understand the organization of the spaCy library and develop a solid understanding of the library itself.

You will then learn the first component of the pipeline : the Tokenizer.

Basic Operations with spaCy II

You will also learn an important linguistic concept : lemmatization, as well as its applications in Natural Language Understanding (NLU).

Then, we will cover in detail the container classes and data structures of spaCy.

We will end the chapter with useful features of spaCy that you will use in daily NLP development.